

Module 7: Lambda Expressions and Linq

Overview

- Anonymous Types
- Anonymous Methods
- Lambda Expressions
- Expression Bodies
- Generic Delegates
- IEnumerable<T> and Extension methods
- LINQ (Language Integrated Query)
- Closure

Anonymous Types

- allow you to create objects **without explicitly defining a class**
- **quick and convenient way** to group related properties into a single object

```
var employee = new { FirstName="John", LastName="Smith" };  
Console.WriteLine(person.FirstName);           // Output: John
```

Compiler generated type

- Type name is generated by the compiler and is not available at the source code level

```
[CompilerGenerated]
internal sealed class <>f__AnonymousType0
{
    private readonly <FirstName>j__TPar<FirstName> i__Field;
    private readonly <LastName>j__TPar<LastName> i__Field;

    public <FirstName>j__TPar FirstName
    {
        get { return this.< FirstName > i__Field; }
    }
    public <LastName>j__TPar LastName
    {
        get { return this.< LastName > i__Field; }
    }
}
```

Anonymous Types

- Method implemented by the compiler
 - Equals()
 - GetHashCode()
 - ToString()
- => can be used in dictionaries and for sorting

Note:

- *Method .ToString() overrides output value of the anonymous type instance in much the same format as it was initialized*

```
{FirstName ="John", LastName ="Smith"}
```

Nested Anonymous Type

- Department is nested into Employee

```
var employee = new {  
    FirstName = "John",  
    LastName = "Smith",  
  
    Department = new {  
        Id = 1,  
        Name = "IT"  
    }  
};
```

Restrictions of Anonymous Types

- Immutable in c# (not in VB.NET)
- Internal only
- Inherited from Object => can be casted only to type Object
- Cannot assign method to anonymous type property

```
var employee = new {  
    FirstName = "John",  
    LastName = "Smith",  
    ToString = new EventHandler(ToString)  
};
```



- Only local variables (var keyword)

Exposing anonymous type

- a) User-defined Class
- b) Using Tuples
- c) Using Reflection

```
object GetData()  
{  
    return new { FirstName = "John", LastName = "Smith"};  
}  
object e1 = GetData();  
string name = e1.GetType().GetProperty("FirstName").GetValue(obj, null);
```

- d) Using Dynamic type

```
dynamic e1 = new { FirstName="Karel", LastName="Novák" };  
name = e1.FirstName;  
name = e1.FName;
```

- e) Using Generics
- f) Records

Lab 07.1 - Anonymous Types



Exercise:

1. Anonymous Type
2. Anonymous Type and method parameters
 1. Reflection
 2. Dynamic type
 3. Generic Cast
3. Generic List<> as anonymous type

Overview

- Anonymous Types
- **Anonymous Methods**
- Lambda Expressions
- Expression Bodies
- Generic Delegates
- IEnumerable<T> and Extension methods
- LINQ (Language Integrated Query)
- Closure

In-line delegate

```
delegate void MyDelegate(int item);

void MyMethod(int item)
{
    Console.WriteLine(item);
}

MyDelegate d = new MyDelegate(MyMethod);
d.Invoke(1);
```

- Anonymous Method also known as in-line delegate

```
delegate void MyDelegate(int item);
MyDelegate d = delegate(int item) { Console.WriteLine(item); }
d.Invoke(1);
```

Anonymous Method

- =method without a name
- a technique to pass a code block as a delegate
- reduce the coding overhead in instantiating delegates because you do not have to create a separate method
- the return type is inferred from signature of delegate where anonymous method is assigned to

```
delegate(int item) { Console.WriteLine(item); }
```

Function as a Black box

```
delegate(int item) { return item + 1; }
```

Input → **BLACK BOX** → **Output**

Anonymous Methods

a) Inline Delegate

```
delegate(int item){ return item + 1; }
```

b) Lambda Expression

Input → **BLACK BOX** → Output

```
(int item) => { return item + 1; }
```

```
item => { return item + 1; }
```

```
item => item + 1
```

Lambda Expressions

- Expression Lambdas

```
item => item + 1
```

- returns the result

- Statement Lambdas

```
item => { return item +1; }
```

- can consist of any number of statements

Delegate System.Action

- A System.Action delegate is a reference to a function that returns no value and no parameter

```
public delegate void Action();
```

- Example:

```
Action d = delegate()  
{  
    Console.WriteLine("Hello World");  
};  
d.Invoke();
```

- or

```
Action d = () => { Console.WriteLine("Hello World"); };  
d.Invoke();
```

Generic Delegate System.Action<>

- A System.Action<> delegate is a reference to a function that returns no value and has 1 to 16 parameters

```
public delegate void Action<T>(T obj);
```

```
public delegate void Action<T1, T2>(T1 arg1, T2 arg2)
```

...

```
public delegate void Action<T1, T2, ..T16>(T1 arg1, T2 arg2, ..T16 arg16)
```

- *Example:*

```
Action<int> d = a => { Console.WriteLine(a + 1); };  
Action<int, int> d = (a, b) => { Console.WriteLine(a + b); };
```

Generic Delegate System.Func<>

- A System.Func<> delegate is a reference to a function that returns a value and has 1 to 16 parameters

```
public delegate TResult Func<TResult>();
```

```
public delegate TResult Func<T1, TResult>(T1);
```

```
public delegate TResult Func<T1, T2, TResult>(T1, T2);
```

...

```
public delegate TResult Func<T1, T2,.. T16, TResult>(T1, T2, ..T16);
```

Example:

```
Func<int, int> d = a => a + 1;  
Func<int, int, int> d = (a, b) => a + b;
```

Delegate System.Predicate<>

- delegate to a function that has
 - one parameter
 - returns true or false



```
public delegate bool Predicate<T>(T obj);
```

Note:

- *Represents the method that defines a set of criteria and determines whether the specified object meets those criteria*

Compare

```
List<int> list = new List<int>() { 10, -20, 30, -40 };

// filter list to newList
List<int> newList = new List<int>();
foreach (int item in list)
{
    if (item > 0) newList.Add(item);
}

return newList;
```

- Optimize using Generic function + Predicate delegate

```
bool Filter(int item)
{
    return item > 0;
}

List<int> newList = list.FindAll(new Predicate<int>(Filter));

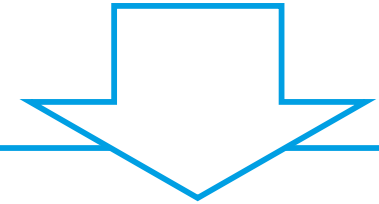
// -- OR --

List<int> newList = list.FindAll(Filter);
```

Compare

```
bool Filter(int item)
{
    return item > 0;
}
```

```
List<int> newList = list.FindAll(Filter);
```



- Anonymous function / Lambda expression

```
List<int> newList = list.FindAll(item => item > 0);
```

Compare Solutions

- Original

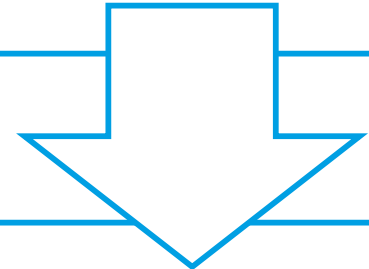
```
List<int> list = new List<int>() { 10, -20, 30, -40 };

// filter list to newList
List<int> newList = new List<int>();
foreach (int item in list)
{
    if (item > 0) newList.Add(item);
}

return newList;
```

- Final

```
List<int> list = new List<int>() { 10, -20, 30, -40 };
List<int> newList = list.FindAll(item => item > 0);
```



Using Lambda Expressions

- Are often used when you need to pass a function as an argument to another function

```
List<int> list1 = list.FindAll(item => item > 0);  
List<int> list2 = list.FindAll(item => item < 0);  
List<int> list3 = list.FindAll(item => item % 2 == 0);  
List<int> list4 = list.FindAll(item => item % 2 != 0);
```

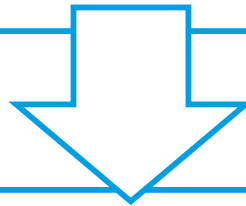


Expression bodies

(>=C# 6.0)

- Expression bodies on method-like members

```
public string SayHello()  
{  
    return "Hello";  
}
```



```
public string SayHello() => "Hello";
```

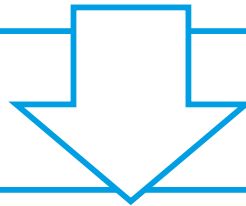
SYNTACTIC SUGAR

Expression bodies

(>=C# 6.0)

- Expression bodies on method-like members

```
public string SaySomething(string something)
{
    return "I Am Saying " + something;
}
```



```
public string SaySomething(string something)
=> "I Am Saying " + something;
```

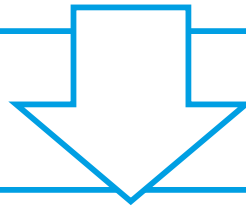
SYNTACTIC SUGAR

Expression bodies

(>=C# 6.0)

- Expression bodies on readonly property-like members

```
public string GetTime  
{  
    get { return DateTime.Now.ToLongTimeString(); }  
}
```



```
public string GetTime  
    => DateTime.Now.ToLongTimeString();
```

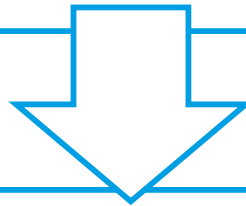
SYNTACTIC SUGAR

Expression bodies

(>=C# 6.0)

- Expression bodies on readonly property-like members

```
public string GetTime  
{  
    get { return DateTime.Now.ToLongTimeString(); }  
}
```



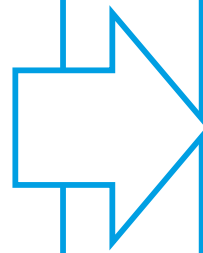
```
public string GetTime  
    => DateTime.Now.ToLongTimeString();
```

SYNTACTIC SUGAR

Expression-Bodied Property Accessors (>= 7.0)

- compact way also for setters

```
private string _name;  
public string Name  
{  
    get  
    {  
        return _name;  
    }  
    set  
    {  
        _name = value;  
    }  
}
```



```
private string _name;  
public string Name  
{  
    get => _name;  
    set => _name = value;  
}
```

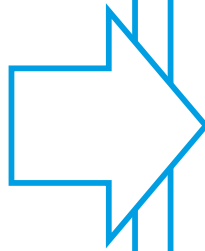
SYNTACTIC SUGAR

Expression-Bodied Constructors (>=C# 7.0)

- compact way also for constructors

```
public class MyClass
{
    string name;

    MyClass()
    {
        name = "John";
    }
}
```



```
public class MyClass
{
    string name;
    MyClass() => name = "John";
}
```

SYNTACTIC SUGAR

Restrictions of Expression-Bodies

- Not allowed
 - block of code
 - Loop statements (i.e. for, foreach, while and do..while are not allowed)
 - Branching statements (if..else, switch) are not allowed
- Allowed conditional operator

```
public string GetName() =>  
(FirstName != null) ? $"{FirstName} {LastName}" : LastName;
```

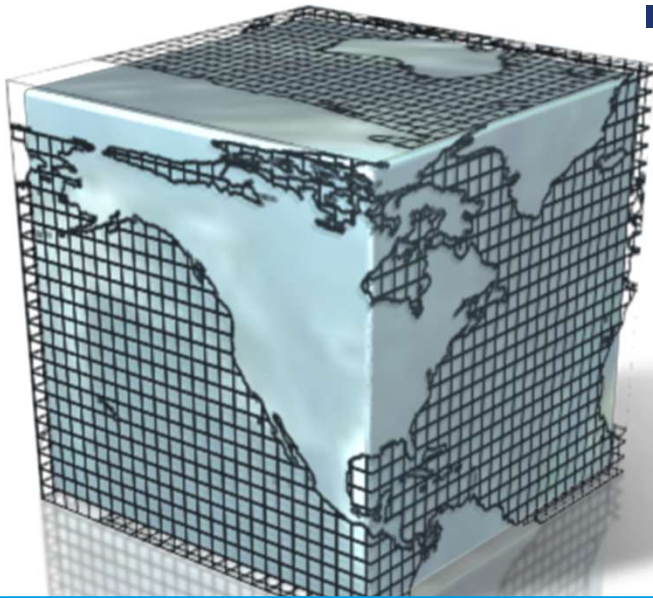
Lab 07.2 - Anonymous Methods and Lambda Expressions



Exercise:

1. Anonymous Method
2. Anonymous Method in method parameter
3. Anonymous Method and Lambda Expression
4. More Lambda expressions
5. Generic delegate Func<>
6. Using expression bodies

Exercise - Anonymous methods



- Perform operations using lamdas
 - 1) add
 - 2) subtract
 - 3) multiply
 - 4) divide

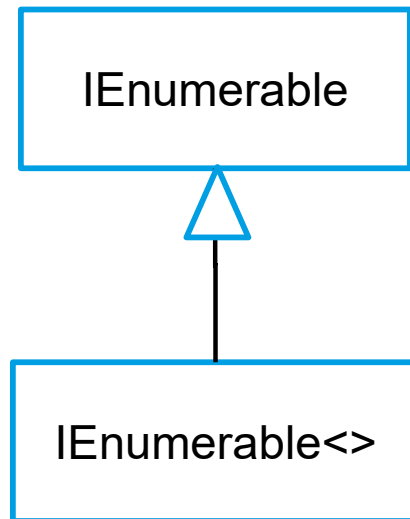
```
// Generic method for executing and logging operations
double PerformOperation(
    double a, double b, Func<double, double, double> operation)
{
    Console.WriteLine($"Performing operation: {a} and {b}");
    double result = operation(a, b);
    Console.WriteLine($"Result of operation: {result}");
    return result;
}
```

Overview

- Anonymous Types
- Anonymous Methods
- Lambda Expressions
- Expression Bodies
- Generic Delegates
- **IEnumerable<T> and Extension methods**
- LINQ (Language Integrated Query)
- Closure

IEnumerable<> Interface

- IEnumerable<> - Generic



```
public interface IEnumerator
{
    Object Current { get; }
    bool MoveNext();
    void Reset();
}
```

```
public interface IEnumerator<T> :
    IDisposable, IEnumerator
{
    new T Current { get; }
}
```

Exploring a Generic Solution for Collections

```
List<int> newList = list.FindAll(item => item > 10);
```

.FindAll() is specific for generic List<>



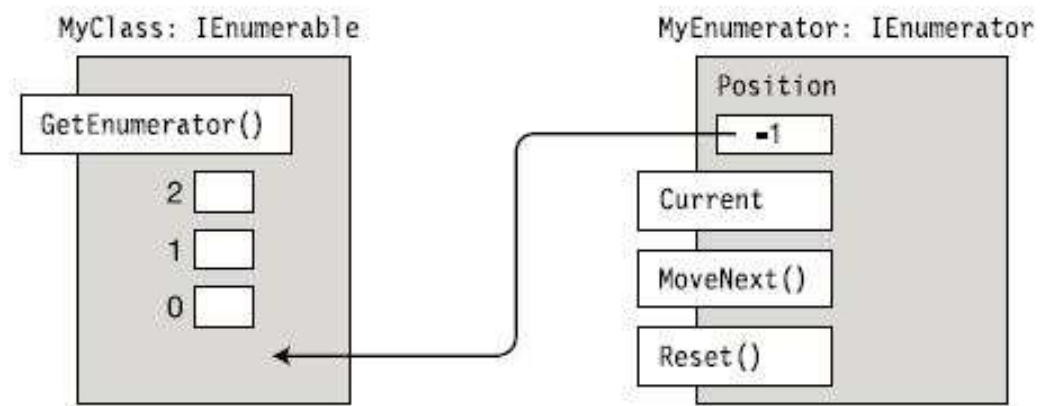
?? Generic solution ??

Lab 07.3 - Extension Methods And IEnumerable



Exercise:

1. Custom collection implementing IEnumerable interface
2. Extension methods to IEnumerable interface



Overview

- Anonymous Types
- Anonymous Methods
- Lambda Expressions
- Expression Bodies
- Generic Delegates
- IEnumerable<T> and Extension methods
- **LINQ (Language Integrated Query)**
- Closure

Exploring a Generic Solution for Collections

```
List<int> newList = list.FindAll(item => item > 10);
```

.FindAll() is specific for generic List<>



?? Generic solution ??



Language INtegrated Query

= set of technologies based on the integration of query capabilities
directly into the C# language

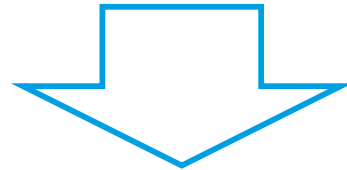
Namespace System.Linq

- Provides classes and interfaces that support queries that use Language-Integrated Query (LINQ).

```
namespace System.Linq
{
    public static partial class Enumerable
    {
        public static IEnumerable<TSource> Where<TSource>(
            this IEnumerable<TSource> source, Func<TSource, bool> predicate) { }
        public static IEnumerable<TSource> First<TSource>(
            this IEnumerable<TSource> source, Func<TSource, bool> predicate) { }
        public static IEnumerable<TSource> Last<TSource>(
            this IEnumerable<TSource> source, Func<TSource, bool> predicate) { }
        public static IEnumerable<TSource> Reverse<TSource>(
            this IEnumerable<TSource> source, Func<TSource, bool> predicate) { }
        public static IEnumerable<TSource> Select<TSource>(
            this IEnumerable<TSource> source, Func<TSource, bool> predicate) { }
    }
}
```

Extension methods of IEnumerable<> Interface

```
using System.Linq;
```

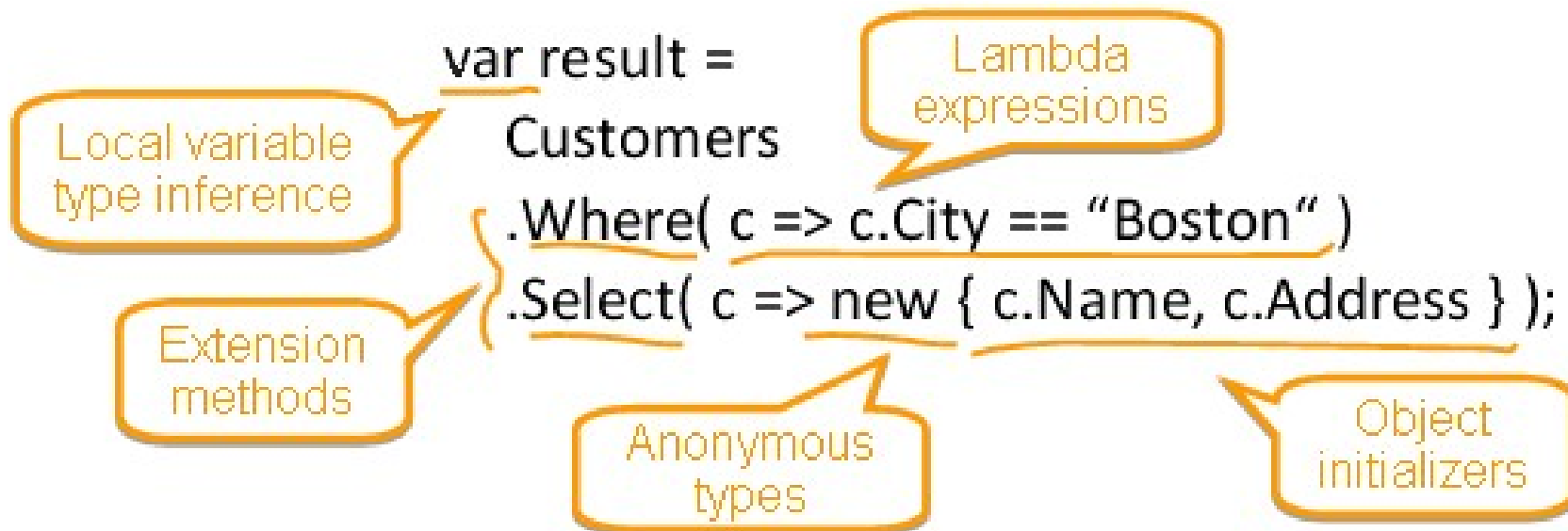


```
var list = trainings.Where(t => t.Name.Contains("ASP")).Select(t => t);
```

or

```
var list = trainings  
    .Where(t => t.Name.Contains("ASP"))  
    .Select(t => t);
```

Is not it all too complex?



Simplify LINQ syntax

- Method syntax

```
var list = trainings
    .Where(t => t.Name.Contains("ASP"))
    .Select(t => t);
```

- Query syntax

```
var list = from t in trainings
            where t.Name.Contains("ASP")
            select t;
```

SYNTACTIC SUGAR

Lab 07.4 - LINQ



- Exercise:
 1. Extension metody z namespace `System.Linq`
 2. Linq and method like syntax
 3. Linq and query like syntax
 4. Using method `ToList()`
 5. Using method `Array.ForEach()`

Overview

- Anonymous Types
- Anonymous Methods
- Lambda Expressions
- Expression Bodies
- Generic Delegates
- IEnumerable<T> and Extension methods
- LINQ (Language Integrated Query)
- **Closure**

Anonymous Methods and local variables

- = variables defined in parent's method body can be referenced from within the anonymous method

```
{  
    int i = 0;  
    MyDelegate d1 = delegate() {  
        i++; // must have access to local variables  
    };  
}
```

- same as

```
{  
    int i = 0;  
    if (true) {  
        i++; // must have access to local variables  
    }  
}
```

Closure

- = variables defined in parent's method body can be referenced from within the anonymous method

```
public static void Main()
{
    int i = 0;
    MyDelegate d1 = delegate() { Console.WriteLine(i); };
    MyClass.MyMethod(d1);
}

class MyClass
{
    public static void MyMethod(MyDelegate d)
    {
        d(); // still must have access to local variables
    }
}
```

Lab 07.5 - Closure



Exercise:

1. Using Closure
2. Practical Use Case

Exercise - Closure



Exercise:

1. Create list of 100 anonymous functions
2. Each function returns "Hello" + index

"Hello 1"
"Hello 2"
...
"Hello 100"

Review

- Anonymous Types
- Anonymous Methods
- Lambda Expressions
- Expression Bodies
- Generic Delegates
- IEnumerable<T> and LINQ (Language Integrated Query)
- Closure